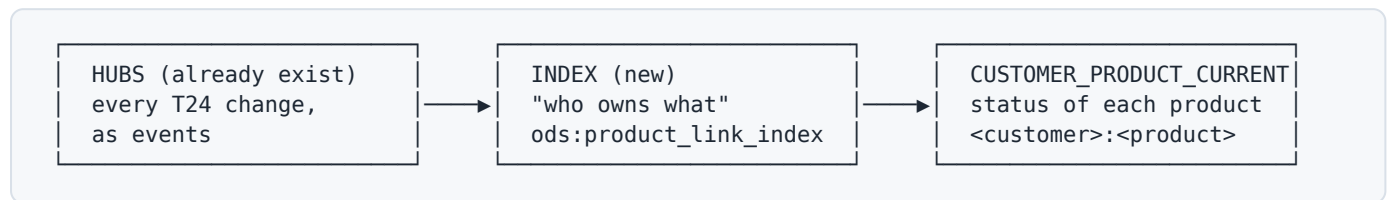


Account Close Cascade

When an account is deleted, close every product under it. A walkthrough in simple pictures, with the code that handles each step.

The three tables



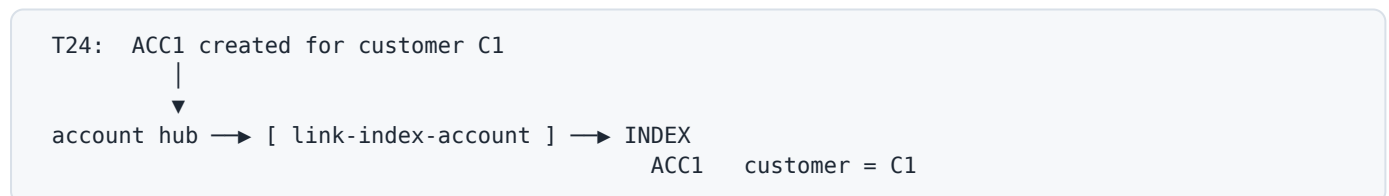
The index holds three kinds of rows. They all sit next to each other because they start with the same id:

ACC1	customer = C1	← the account's customer
ACC1:MM1	l = ''	← "MM1 is under ACC1"
MM1	owner = ACC1	← "MM1 belongs to ACC1 right now"

Code behind it

- `LinkIndex.java` defines the index row shapes (`ACC1` , `ACC1:MM1` , `MM1`) and the cell names `customer` , `owner` and `l` .
- `hbase-ddl.hbase` creates the index table.
- `account-close-cascade.yaml` connects the jobs together.

Case 1 A new account is created, or an account is updated



The job only saves the account's customer. We need it later to find the account's rows in `CUSTOMER_PRODUCT_CURRENT` .

Code behind it: no new Java

- **What to write:** a short SQL query inside `account-close-cascade.yaml` (entry `link-index-account`): "take the account id and its customer".
- **Who runs it:** the existing `HubSqlPipeline.processBatch` . It reads each batch of changes from the account hub and runs the query.
- **Who writes to HBase:** the existing sink, `InternalRowPutMapper.toPut` . Each map entry becomes one cell, and the newest record wins.

Case 2 A new money-market contract is opened

T24: MM1 opened on account ACC1

↓
MM hub → [link-index-money-market] → INDEX
ACC1:MM1 l = '' (listed under ACC1)
MM1 owner = ACC1 (owned by ACC1)

The job writes two rows: one says MM1 is listed under ACC1, the other says ACC1 is MM1's owner. An ordinary update to MM1, such as a new amount, writes the same two rows again, so nothing changes.

Code behind it: no new Java

- **What to write:** `link-index-money-market.sql`. Its last 3 lines produce the two rows: `concat(owner, ': ', child)` for the link, and `map('owner', owner)` for the owner.
- **Who runs it and writes it:** the same `HubSqlPipeline.processBatch` and `InternalRowPutMapper.toPut` as in Case 1.

Case 3 A money-market contract moves to another account

T24: MM1 moves from ACC1 to ACC2

↓
MM hub → [link-index-money-market] → INDEX
ACC1:MM1 l = '' ← old row stays
ACC2:MM1 l = '' ← new row
MM1 owner = ACC2 ← overwritten: now ACC2

The old row `ACC1:MM1` stays, because the index never deletes anything. But MM1's **owner** now says ACC2, and the owner is what decides which account can close it.

Code behind it: the same SQL as Case 2

- `owner` gets the new value because the newest record wins in `InternalRowPutMapper.toPut`. It uses `hub_version` as the cell's timestamp, so an old record replayed late can't bring `ACC1` back.

Case 4 An account is deleted (the main case)

```
T24: ACC1 deleted
      |
      v
account hub (delete marker) → [ account-delete-cascade ]
```

Code behind it: `DeleteCascadePipeline.java`, the new pipeline type `delete-cascade`

- `run` starts a stream on the account hub and handles every batch.
- `deletedNodes` keeps only the delete markers (`op = D`) and returns the deleted account ids, for example `[ACC1]`.

Step 1: Who is the customer?

```
read ACC1 → customer = C1
```

Code behind it

- `CloseCascade.walk` is the brain of the process. It asks `store.read`.
- `HBaseStore.read` does one batched **multi-Get** and returns the `owner` and `customer` of each id.

Step 2: What is under ACC1?

```
scan ACC1:* → MM1, MM2
```

Code behind it

- `HBaseStore.children` does one **prefix scan**, from `ACC1:` up to `ACC1;`. `LinkIndex.linksStart` / `linksStop` build those two boundaries.
- It reads only the row keys and splits each one into account and product.

Step 3: Do they still belong to ACC1?

```
read MM1 → owner = ACC1 ✓ close it
read MM2 → owner = ACC2 ✗ it moved, skip it
```

Code behind it

- `walk` reads the products found with `store.read`, the same multi-Get as in Step 1.
- **The single line that decides:** `c.parent.equals(row.owner)` in `CloseCascade.java` means "close only if its owner is the account we found it under".

Step 4: Close them

```
write CUSTOMER_PRODUCT_CURRENT
C1:ACC1 s = CLOSED rsn = ACCOUNT_DELETED
C1:MM1 s = CLOSED rsn = ACCOUNT_DELETED
```

Code behind it

- `HBaseStore.write` builds one HBase write per product: `cid`, `pid`, `s = CLOSED`, `rsn = ACCOUNT_DELETED`, `m`, `ts`, `src`.
- `customerProductKey` builds the row key `C1:MM1`.
- If the account had no customer, nothing is written and a warning is logged.

In pictures:

```
ACC1 (deleted)
├─ MM1  owner = ACC1  → CLOSED ✓
├─ MM2  owner = ACC2  → Left open (it moved) ✗
```

Case 5 Products that have their own children, like limits

Sub-products are stored the same way, one level lower:

```
ACC1:L1      ← limit L1 under ACC1
L1           owner = ACC1
L1:L1.01     ← sub-limit under L1
L1.01        owner = L1
```

The cascade repeats Steps 2-3 one level at a time, going down at most 5 levels:

```
ACC1 (deleted)
├─ L1      → CLOSED
│   └─ L1.01 → CLOSED
```

Code behind it: the loop inside `CloseCascade.walk`

- The closed products of one level become the parents of the next level. Steps 2-3 then repeat, up to `maxDepth` (5 in the catalog).
- A `seen` list makes sure the same id is never handled twice, even if the data contains a loop.

The whole map in one picture

T24 change	Job (catalog entry)	Code that does the work
Account created	→ link-index-account	→ HubSqlPipeline + sink (existing)
MM opened/moved	→ link-index-money-market	→ HubSqlPipeline + sink (existing)
Account deleted	→ account-delete-cascade	→ DeleteCascadePipeline.run (new) └─ deletedNodes which accounts? └─ CloseCascade.walk what to close? └─ HBaseStore.read multi-Get └─ HBaseStore.children prefix scan └─ HBaseStore.write mark CLOSED

Where the tests are

- CloseCascadeTest checks the rules of walk : a moved product isn't closed, loops are handled, and the depth limit works.
- DeleteCascadeStoreTest runs the real HBaseStore against the devkit's in-process fake HBase.
- AccountCloseCascadeCatalogTest checks that the catalog and the SQL match the code.

In one sentence: every product change keeps the index up to date. When an account is deleted, the cascade looks up everything under that account in the index, skips anything that has moved to another account, and marks the rest as **CLOSED** in CUSTOMER_PRODUCT_CURRENT .